# INF201 - Lecture 10

9th Nov 2026

## Topics

- Object-oriented programming
  - Fundamentals
  - Namespaces
  - Copying items
  - Best practices for writing classes
- Adding command line arguments to your scripts

## Object-orientation: Fundamentals

### Idea 1: Combine data and operations into new data types

- A (user-defined) data type is a *class*
- Each *object* is an *instance* of a class
- Classes and objects have *attributes* and *methods*
  - *Methods* are functions operating on objects
  - *Attributes* are data (variables) contained in objects

In [ ]:
```
from geometries import Circle

c1 = Circle(center=0, radius=1)
type(c1)  # data type: geometries.Circle
```

In [ ]:
```
type(Circle)
```

In [ ]:
```
vars(c1)  # attributes
```

Behind the scenes: `vars(c1)` calls the special method `__dict__` of the object `c1`:

In [ ]:
```
vars?
```

In [ ]:
```
c1.__dict__
```

Note: You should generally not call the special methods (dunders) when you use the classes, they are for internal use only. This is why we would inspect the object attributes by `vars()` instead of `__dict__`.

### Idea 2: Allow modification and extension of data types

- *Inheritance*: Define a *subclass* (or child class) based on a *superclass* (or parent class) and adapt or extend it

- Inheritance allows us to write more specialised classes based on general ones
  - Can create deep hierarchies here, but the recommendation is to keep it shallow

### Trace the hieracrchy using `__base__`

In [ ]:
```
from pets_updated import Dog, Pet

print(Dog.__base__)  # __base__ show us the parent class
```

In [ ]:
```
print(Pet.__base__)
```

In [ ]:
```
print(object.__base__)  # object is the top of the hierarchy
```

### Who gets called when?—Method resolution order

Define three classes `A > B > C` where each subclass redefines one method.
(We write this in a compact way, without any doc-strings and type hints for now.)

In [ ]:
```
class A:
    def f(self): return "A.f()"
    def g(self): return "A.g()"
    def h(self): return "A.h()"


class B(A):
    def g(self): return "B.g()"


class C(B):
    def h(self): return "C.h()"
```

Visualise this code with Pythontutor

Create one instance per class.

In [ ]:
```
a = A()
b = B()
c = C()
```

Call methods on `A` instance

In [ ]:
```
print(a.f(), a.g(), a.h())
```

Call methods on `B` instance

In [ ]:
```
print(b.f(), b.g(), b.h())
```

Call methods on `C` instance

In [ ]:
```
print(c.f(), c.g(), c.h())
```

- Methods are looked up from bottom and up
  1. instance
  2. class
  3. superclasses in ascending order
  4. `object`
- Lookup order for a class is available through the attribute `__mro__` or method `mro()` (method resolution order)

In [ ]:
```
A.__mro__
```

In [ ]:
```
B.__mro__
```

In [ ]:
```
C.__mro__
```

For a more complex example, where a class has two superclasses (parents):

In [ ]:
```
class D(A):
    pass


class E:
    def g(self): return "E.g()"
    pass


class F(D, E):
    pass


f = F()
print(f.f(), f.g(), f.h())
```

In [ ]:
```
F.__mro__
```

- For multiple inheritance, MRO will be determined like this:
  - A class appears before its parents.
  - Parent classes appear in the order listed.
  - Each class appears only once, preserving relative order.
- If Python cannot determine a consistend MRO, it will fail with a `TypeError`
- `super()` in class definitions refers to the next class in the MRO

## Idea 3: Expose an interface, hide the implementation

- *Interface*: Methods "advertised" for public use
  - Provides access to functionality of the class
  - Should be stable over time
  - Should be well documented
  - Should be well tested
- *Implementation*: Does the actual work

    - User should not need to know about it
    - Can change at any time (refactoring)
    - Changes should not affect behavior
- Also know as *encapsulation*

In [ ]:
```
from geometries import Rectangle

Rectangle??
```

We can change the *implementation* of the rectangle without changing the *interface*

In [ ]:
```
class Rectangle:
    def __init__(self, lower_left: tuple[int | float, int | float], upper_right:
        """Create rectangle based on lower left and upper rigth coordinates"""
        self.width = upper_right[0] - lower_left[0]
        self.height = upper_right[1] - lower_left[1]

    def area(self) -> int | float:
        """Calculates the area of the rectangle"""
        return self.width * self.height
```

We can further protect the height and width attributes, by indicating with `_`

In [ ]:
```
class Rectangle:
    def __init__(self, lower_left: tuple[int | float, int | float], upper_right:
        """Create rectangle based on lower left and upper rigth coordinates"""
        self._width = upper_right[0] - lower_left[0]
        self._height = upper_right[1] - lower_left[1]

    def area(self) -> int | float:
        """Calculates the area of the rectangle"""
        return self._width * self._height
```

... and create properties for viewing the protected attributes

In [ ]:
```
class Rectangle:
    def __init__(self, lower_left: tuple[int | float, int | float], upper_right:
        """Create rectangle based on lower left and upper rigth coordinates"""
        self._width = upper_right[0] - lower_left[0]
        self._height = upper_right[1] - lower_left[1]

    def area(self) -> int | float:
        """Calculates the area of the rectangle"""
        return self._width * self._height

    @property
    def width(self) -> int | float:
        """View-only property for rectangle width"""
        return self._width

    @property
    def height(self) -> int | float:
        """View-only property for rectangle width"""
        return self._height
```

In [ ]:
```
r1 = Rectangle((1, 1), (5, 5))
print(f"Width: {r1.width}")
print(f"Height: {r1.height}")
```

# Names, Namespaces, and Scopes

## Why worry about names?

1. Programs execute functions to manipulate data.
2. Data and functions are stored as sequences of bits in memory.
3. We need *names* to refer to data and functions in our programs.
4. In large programs
    - the same name may be used or different purposes in different places
    - it is impossible to keep an overview over all names
    - E.g.: what do you get if you run `from xyz import *`?
        - (Wildcard imports like this is generally a bad idea, don't do it)
5. Solution: *namespaces* and *scoping rules*

### Namespaces (navnerom)

Namespaces help to keep names organized.
A namespace is a 'place' with a set of names and their corresponding objects. (From here)

### Scoping rules (regler for gyldighetsområder)

Scoping rules define which namespace applies in each part of a program.

## How do we bind names to objects in Python?

Operation | Example | Name bound :- | :- | - Assignment | `x = 2` | `x` Function definition | `def f(): pass` | `f` Class definition | `class A: pass` | `A` Module import | `import math` | `math` || `import math as m` | `m` || `from math import sin` | `sin`

### Definitions are statements

Definitions are statements in Python programs. They are executed just as all other statements.

### Presentation on topic

- See also https://nedbatchelder.com/text/names1.html

## Where are names bound?

```
class Friend:

    greeting = "Hi, "

    def __init__(self, name):
        self.name = name

    def greet(self):
        text = Friend.greeting + self.name
        print(text)

joe = Friend("Joe")
```

Bound in namespace: module, class, function, instance

### Name-binding rules

- When a name is bound, it is registered in *exactly one namespace*
- Available namespaces
  - `__builtin__` namespace of Python interepreter
  - each module has a namespace (each imported `*.py` file)
  - each class has a namespace
  - each class *instance* has a namespace
  - each function *invocation* has a namespace
- In which namespace is a name registered?
  - In the namespace of the *innermost scope*
  - Inside a list comprehension: in the comprehension's namespace
  - Inside function definitions: in the function invocation's namespace
  - Inside class definitions: in the class' namespace
  - Otherwise, in the module's namespace

In [ ]:
```
list = [x for x in range(5)]
x  # Only defined within the list comprehension
```

In [ ]:
```
__builtin__?
```

Technically, the Python's built-in functions are all in the builtin module

In [ ]:
```
# Jupyter magic for showing all names in current notebook
# List all available magics by %lsmagic
%whos
```

Note that the classes we have defined are listed here, but not the methods inside the classes.
These methods belong to the namespace of the class, not the notebook.

## Where are names looked up?

```
class Friend:

    greeting = "Hi, "

    def __init__(self, name):
        self.name = name

    def greet(self):
        text = Friend.greeting + self.name
        print(text)

joe = Friend("Joe")
```

Lookup in: L: local (function), G: global (module), B: builtin

## Name-lookup rules

- When we use a name, Python must loop up the name and find the object it refers to
- In which namespace does Python look?
- **LEGB rule** (Mark Lutz, *Learning Python*)
  - **L—Local:** Namespace of the function invocation currently executing
  - **E—Enclosing:** Namespaces of all functions enclosing the definition of the current function (ignore for now)
  - **G—Global:** Namespace of module in which the current function *was defined*
  - **B—Builtin:** Namespace of Python builtins
- Exceptions can be forced with `global` and `nonlocal` keywords (avoid for now)

## Attribute/method lookup with "dot"

- Modules, classes, and instances have attributes and methods
- Their names are bound inside module, class, instancance namespace
- Accessible through the dot-operator:

In [ ]:
```
class Friend:

    greeting = 'Hi, '

    def __init__(self, name: str) -> None:
        """Create Friend object with attribute name"""
        self.name = name

    def greet(self) -> None:
        """Print string greeting self"""
        print(Friend.greeting + self.name)


joe = Friend("Joe")
print(joe.name)
```

In [ ]:
```
joe.name = "Joe Doe"  # change the name attribute
joe.greet()
```

In [ ]:
```
Friend.greeting = "Hello, "  # change the greeting in the class Friend
joe.greet()
```

### Attribute lookup in instances vs classes

```
class Friend:
    def __init__(self, name):
        self.name = name

    def greet(self):
        print("Hi, " + self.name)

joe = Friend("Joe")
print(joe.name)
joe.greet()
```

Found in: instance, class

### Pitfall: Duplicate attribute names

- Python uses the same namespace for methods and data attributes
- Names are looked up in the instance namespace first, then in the class namespace
- This may lead to surprises when using the same name in multiple places

In [ ]:
```
class Friend:

    def __init__(self, name: str) -> None:
        """Create Friend with name"""
        self.name = name

    def greet(self) -> None:
        """Print greeting"""
        print('Hi,', self.name)

    def name(self) -> None:
        """Print statement of name"""
        print('Your name is', self.name)
```

In [ ]:
```
joe = Friend('Joe')
joe.greet()
```

In [ ]:
```
joe.name()
```

- What happended here?
  - Lookup starts with the instance, where we find the string stored in `self.name``
  - Then Python tries to call this string as a function because of the `()` after `joe.name`
  - Lookup stops at first match and thus never sees the method `name(self)` in this case

Visualised with Pythontutor

# Copying objects

Objects with custom classes comes with the same potential pitfall for copying that objects of built-in classes (such as lists) do.

In [ ]:
```
class ClassName:
    pass


a = ClassName()
a.x = 10  # We create an instance attribute
b = a  # a and b are two names for the same object
b.x = 20
print(a.x, b.x)
```

- `a` and `b` refer to the same object, so changing `b` also changes `a`
- Visualise this with PythonTutor

Sidenote: If you want to prevent the user from setting instance attributes not defined in the class you can use `__slots__`.
Read more about them in Python's documentation on slots and in chapter 3 of Advanced Guide to Python 3 Programming.

## Python's `copy` module

- Provides functions to copy objects

In [ ]:
```
import copy

c = copy.copy(a)
c.x = 50
print(a.x, b.x, c.x)
```

In [ ]:
```
s = ClassName()
s.m = 'Nice weather today.'
t = copy.copy(s)
print(hex(id(s)), hex(id(t)))
print(hex(id(s.m)), hex(id(t.m)))
```

- `s` and `t` have *different* `id()`: they are *different* `S` *instances*
- `s.m` and `t.m` have the same `id()`: they are the *same string instance*
- `copy.copy()` is a *shallow copy*: `t` is a new instance with its own namespace, but the names refer to the same objects as in `s`
- Assignment to a member re-binds the name to a new string object

In [ ]:
```
t.m = 'The forecast for tomorrow is also nice.'
print(hex(id(s)), hex(id(t)))
print(hex(id(s.m)), hex(id(t.m)))
```

## Copying object with mutable members: deep copy

- Mutables: lists, dictionaries, objects of most classes
- What does shallow copy mean for objects with mutable members?

In [ ]:
```
u = ClassName()
u.m = [1, 2, 3]
v = copy.copy(u)
u.m.append(4)
print(u.m, v.m)
print(hex(id(u.m)), hex(id(v.m)))
```

- Lists are changed in *both* `u` and `v` because their `m` refers to the same list object
- Solution: *deep copy* (as we used for lists earlier)

In [ ]:
```
w = copy.deepcopy(u)
print(u.m, v.m, w.m)
print(hex(id(u.m)), hex(id(v.m)), hex(id(w.m)))
```

**Forgetting the difference between shallow and deep copy is a common source of errors in Python programs!**

# Recommendations and best practices for writing classes

- Use doc-strings and type hints
  - The same way as you would for functions
- Keep clases small
  - Each class should have a single responsibility
- Place class attributes at the top, followed by `__init__`, then public methods, then private helpers
- The initialiser should not allow you to create an object with "illegal" attributes
  - For example: a circle with a negative radius, a dice with only one side
- Implement methods or overload existing methods only when it makes sense
  - For example: only implement `__lt__` (less than) and other comparisons if there is a logical way to sort objects of this class
  - Most classes should (at least) have `__init__` and `__repr__` (help with debugguing)
- Attributes that the user does not need to access, should be protected
  - If you want a viewer or setter for the protected attribute, use the `@property` decorator
- Keep inheritance hieracrhies shallow
- If both options make sense: Chose composition over inheritance

# Including command-line options

- Sometimes you want to be able to run your code from the command-line
  - For example: You are training an AI algorithm on a supercomputer that only has command-line
  - Or maybe you are running a simulation and want to test it with lots of different stating points

- In such cases it is useful to be able to pass the needed arguments using the command-line

```
python my_script.py --one_argument 10 --other_argument "something"
```

- Python has a built-in package `argparse` for including command-line arguments
  - argparse
  - Tutorial
- Overall idea
  - Build a parser by adding options we want to support
  - Can also provide default values and help texts
  - Then run parser to evaluate command line arguments
  - Finally extract values from parser
- Let's look at `demo.py` for a very simple example
- We exploit object-orienter programming here
  - We create an instance of class `ArgumentParser``
  - We use its `add_argument()` method to add the options we want
  - We use its `parse_args()` method to analyse the command line and get and `args` instance containing all options
  - We can now look up the options as fields of `args`, e.g., `args.name``

In [ ]:
```
import argparse

argparse.ArgumentParser?
```